Programming for AI, Assignment 3

NumPy, Pandas and Matplotlib on real measurement data

National University of Computer and Emerging Sciences | Fall 2026 | Total Marks: 100

Instructions

- Work only in this notebook. Fill the cells marked `# YOUR CODE HERE` and the markdown cells marked **ANSWER.**
- Allowed libraries: numpy, pandas, matplotlib. Do not change the names of the data files.
- Use a fixed seed (`np.random.seed(42)`) wherever you need random numbers.
- Every interpretation must refer to actual numbers from your own output. Generic statements get zero.
- Your notebook must run top to bottom with Kernel > Restart and Run All. Notebooks that fail halfway lose 10 marks.
- Q1 and Q2 must be answered by reasoning first. You may run code afterwards only to check yourself, but your written answer must explain the reasoning.
- Submit through GitHub as described in the PDF.

Question	Topic	Marks
1	Code tracing	12
2	Debugging	10
3	Sensor cube (NumPy)	18
4	Weather stations (Pandas)	20
5	Student performance (Pandas + NumPy)	15
6	Dashboard (Matplotlib)	15
7	Surfaces and animation	10

In [101…

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib.gridspec import GridSpec
from matplotlib.animation import FuncAnimation, PillowWriter
np.random.seed(42)
print(np.__version__, pd.__version__)
```

```
2.5.4 3.1.1
```

Question 1: Code tracing (12 marks, 3 each)

Predict the exact printed output of each block **without running it**. Write shapes as tuples and mention dtype where it matters. Then add one or two sentences saying which line is the trap.

1.a

```
import numpy as np
a = np.arange(24).reshape(2, 3, 4)
b = a[:, ::-1, -2:]
b[0, 0, 0] = 99
print(b.shape)
print(a[0, 2, 2])
print(a[:, 1, :].sum(axis=0))
print(b.T.shape)
```

ANSWER: (2, 3, 2) 99 [20 21 22 23] (2, 3, 2) The slicing creates a view of the original array, so changing a value in b also changes a. The trap is .sum(axis=0), which adds the values across both layers and .T reverses the axes without changing the shape like in this case

1.b

```
import numpy as np
x = np.array([10, 20, 30, 40, 50, 60])
idx = np.array([-1, 0, -2, 2])
y = x[idx]
y[0] = 0
print(x)
z = x[1:4]
z[:] = 7
print(x)
print(x[[-1, -1, 0]])
```

ANSWER: [10 20 30 40 50 60] [10 7 7 7 50 60] [60 60 10] The trap is that fancy indexing creates a copy, so changing y does not affect the original array x. However, slicing creates a view, so changing z also changes the original array. Negative indexing selects elements from the end of the array.

1.c

```
import numpy as np
s = np.array([3.9, -3.9, 2.5, 3.5, -0.5])
print(s.astype(int))
print(np.round(s))
print(np.array(['1', '2.5']).astype(float).dtype)
print((s > 0).sum(), (s > 0).any(), (np.abs(s) > 3).all())
```

ANSWER: [3 -3 2 3 0] [4. -4. 2. 4. -0.] float64 3 True False Explanation:

The trap is that astype(int) removes the decimal part instead of rounding, while np.round() rounds exact half values to the nearest even integer. Also, any() returns True if at least one condition is True, whereas all() returns True only if every condition is True.

1.d

```
import numpy as np
A = np.arange(6).reshape(2, 3)
B = np.array([[10], [20]])
C = np.array([1, 2, 3])
print(A + B + C)
print(A @ C)
print(A * C)
print(np.cumprod(C + 1))
print(A.sum(axis=0) - A.sum(axis=1)[0])
```

ANSWER: [[11 13 15] [25 27 29]] [8 26] [[0 2 6] [3 8 15]] [2 6 24] [0 2 4] NumPy automatically repeats compatible arrays to perform calculations. Also, @ performs matrix multiplication while * performs element-wise multiplication. The cumprod() calculates running products, and axis=0 and axis=1 control whether columns or rows are summed.

Question 2: Debugging (10 marks, 2 each)

For each snippet: (i) name the exact exception or the silent logical bug, (ii) explain why in one or two sentences, (iii) write corrected code in the code cell below it, (iv) state the corrected output or shape.

2.a

```
A = np.array([[1, 2, 3], [4, 5, 6]])
B = np.array([[1, 2, 3], [4, 5, 6]])
print(np.dot(A, B))
```

Intended: the 2x2 matrix of dot products between rows of A and rows of B.

ANSWER: ValueError

Reason: The shapes of A and B are (2,3) and (2,3). Matrix multiplication is not possible because their inner dimensions are different. Transposing B makes the dimensions

In [102…

```
A = np.array([[1, 2, 3], [4, 5, 6]])
B = np.array([[1, 2, 3], [4, 5, 6]])
print(A @ B.T)
```

```
[[14 32]
 [32 77]]
```

2.b

```
scores = np.arange(12, dtype=float).reshape(3, 4)
centered = scores - scores.mean(axis=1)
print(centered)
```

Intended: subtract each row's own mean from that row.

ANSWER: error: ValueError

Reason: The shape of scores is (3, 4), while scores.mean(axis=1) has shape (3,). These shapes are not compatible for broadcasting, so a ValueError occurs. [[-1.5 -0.5 0.5 1.5]

[-1.5 -0.5 0.5 1.5] [-1.5 -0.5 0.5 1.5]] keepdims=True keeps the row means in shape (3, 1), allowing NumPy to subtract each row's mean from its own elements correctly.

In [103…
```
scores = np.arange(12, dtype=float).reshape(3, 4)
centered = scores - scores.mean(axis=1, keepdims=True)
print(centered)
```

```
[[-1.5 -0.5  0.5  1.5]
 [-1.5 -0.5  0.5  1.5]
 [-1.5 -0.5  0.5  1.5]]
```

2.c

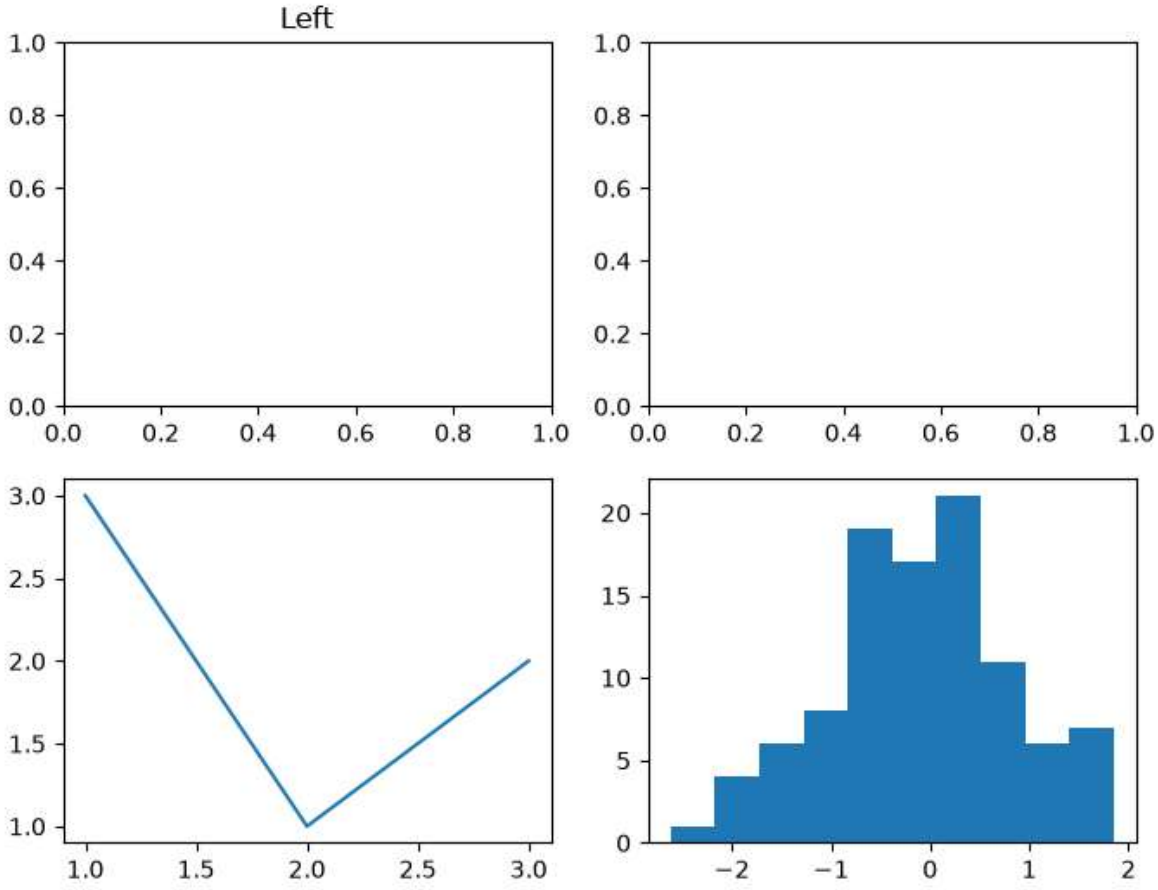
```
fig, ax = plt.subplots(2, 2, figsize=(8, 6))
ax[1].plot([1, 2, 3], [3, 1, 2])
ax[0, 0].set_title('Left')
ax[2, 0].hist(np.random.randn(100))
```

Intended: line plot in the bottom-left panel and histogram in the bottom-right panel.

ANSWER: The first line causes an AttributeError because ax[1] contains multiple Axes objects, not a single plotting area. The third line causes an IndexError because row index 2 does not exist in a 2×2 subplot array.Expected output: A 2×2 figure with a line plot in the bottom-left panel, a histogram in the bottom-right panel, and the title Left in the top-left panel.

In [104…
```
# YOUR CODE HERE (corrected version)
fig, ax = plt.subplots(2, 2, figsize=(8, 6))
ax[1, 0].plot([1, 2, 3], [3, 1, 2])
ax[0, 0].set_title('Left')
ax[1, 1].hist(np.random.randn(100))
```

Out[104…
```
(array([ 1.,  4.,  6.,  8., 19., 17., 21., 11.,  6.,  7.]),
 array([-2.6197451 , -2.17254278, -1.72534045, -1.27813812, -0.83093579,
        -0.38373346,  0.06346887,  0.5106712 ,  0.95787353,  1.40507586,
         1.85227818]),
 <BarContainer object of 10 artists>)
```



2.d

```
marks = np.array([[35., 80., 55.], [20., 90., 45.]])
first = marks[:, 0]
first[first < 40] = 40          # curve for display only
print(marks.mean(axis=0))
```

Intended: the original marks must remain unchanged, the curved copy is used only for display.

ANSWER: logical error.

Reason: marks[:, 0] creates a view of the original array. Therefore, changing values in first also changes the original marks, which affects the calculated mean. [27.5 85. 50.] Slicing creates a view, so .copy() is needed when we want to modify selected values without changing the original array.

In [105…

```
# YOUR CODE HERE (corrected version)
marks = np.array([[35., 80., 55.], [20., 90., 45.]])
first = marks[:, 0].copy()
first[first < 40] = 40
print(marks.mean(axis=0))
```

```
[27.5 85.  50. ]
```

2.e

```
x = np.linspace(0, 1, 50)
y = np.linspace(0, 1, 30)
X, Y = np.meshgrid(x, y)
```

```
Z = np.sin(3 * x) * np.cos(2 * Y)
fig, ax = plt.subplots()
im = ax.pcolormesh(X, Y, Z)
plt.colorbar()
```

Intended: heatmap of sin(3x)cos(2y) with a working colorbar and no shape error.

ANSWER: ValueError due to incompatible shapes.

Reason: X and Y both have shape (30, 50), but x has shape (50,) NumPy cannot broadcast shapes (50,) and (30, 50) as intended. We should use X instead of x. Expected output: A heatmap with 30 rows and 50 columns, showing the values of (sin(3x) cos(2y)), along with a working colorbar.

In [106…

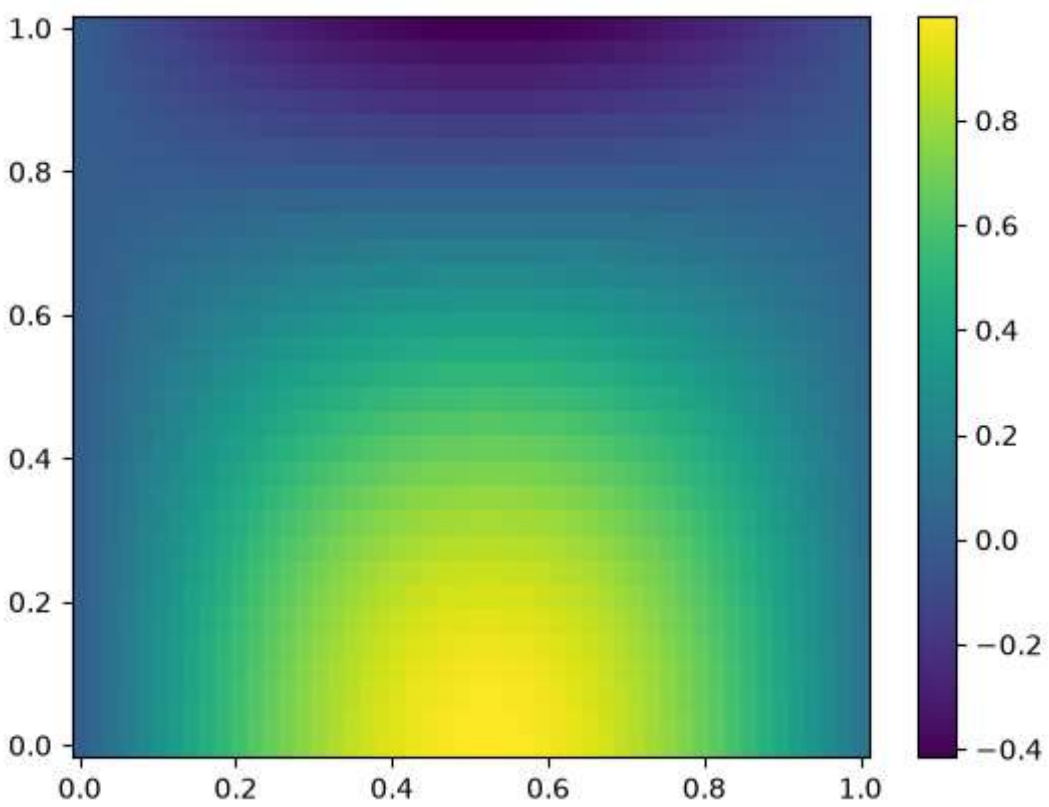
```
# YOUR CODE HERE (corrected version)
x = np.linspace(0, 1, 50)
y = np.linspace(0, 1, 30)
X, Y = np.meshgrid(x, y)

Z = np.sin(3 * X) * np.cos(2 * Y)

fig, ax = plt.subplots()
im = ax.pcolormesh(X, Y, Z)
plt.colorbar(im, ax=ax)
```

Out[106…

```
<matplotlib.colorbar.Colorbar at 0x14c800a7380>
```



Question 3: Sensor cube (18 marks)

`data/sensor_cube.npy` holds temperature readings from 6 sensors over 10 days, 24 hours per day. Axes are (day, hour, sensor). `data/sensor_meta.npz` holds `offsets` (calibration per sensor), `sensor_ids` and `days`. The logger writes -999 when a

reading fails. Some other readings are physically impossible for this site (below 0 or above 60 degrees C).

3a (3 marks) Load both files. Print the shape, dtype and size of the cube, list the arrays stored in the npz archive and print `sensor_ids`. Report how many readings equal -999 and how many other readings are impossible. Why is -999 more dangerous than NaN if you take a mean directly?

In [107…
```
# YOUR CODE HERE

import numpy as np
cube = np.load('data/sensor_cube.npy')
meta = np.load('data/sensor_meta.npz')
print("Shape:", cube.shape)
print("Dtype:", cube.dtype)
print("Size:", cube.size)
print("Arrays:", meta.files)
print("Sensor IDs:", meta['sensor_ids'])
failed = np.sum(cube == -999)
impossible = np.sum(((cube < 0) | (cube > 60)) & (cube != -999))
print("Failed readings:", failed)
print("Other impossible readings:", impossible)
```

```
Shape: (10, 24, 6)
Dtype: float64
Size: 1440
Arrays: ['offsets', 'sensor_ids', 'days']
Sensor IDs: ['T1' 'T2' 'T3' 'T4' 'T5' 'T6']
Failed readings: 27
Other impossible readings: 2
```

3b (4 marks) Make a float copy of the cube (not a view) where every failed or impossible reading is NaN. Prove with `np.shares_memory` that your copy is independent. Then apply calibration by adding `offsets` using broadcasting only, no loops. State the shape of the broadcast operand and why it aligns with the last axis. Write the check that the original file array is untouched.

In [108…
```
# YOUR CODE HERE
original = cube.copy()
clean_cube = cube.astype(float).copy()
clean_cube[(clean_cube == -999) | (clean_cube < 0) | (clean_cube > 60)] = np.nan
print('Independent copy:', not np.shares_memory(cube, clean_cube))
offsets = meta['offsets']
calibrated = clean_cube + offsets
print('Broadcast operand shape:', offsets.shape)
print('Calibrated shape:', calibrated.shape)
print('Original unchanged:', np.array_equal(cube, original))
```

```
Independent copy: True
Broadcast operand shape: (6,)
Calibrated shape: (10, 24, 6)
Original unchanged: True
```

3c (4 marks) Without loops compute: (i) the mean temperature of each sensor over all days and hours, (ii) the mean daily profile of each hour averaged over days and sensors

(shape 24), (iii) the hottest (day, sensor) pair using daily means. NaN must not poison the result. State the axis you used for each and why.

In [109…
```
# YOUR CODE HERE
sensor_means = np.nanmean(calibrated, axis=(0, 1))
hour_profile = np.nanmean(calibrated, axis=(0, 2))
daily_means = np.nanmean(calibrated, axis=1)
hot_day, hot_sensor = np.unravel_index(np.nanargmax(daily_means), daily_means.sh
print('Mean per sensor:', sensor_means)
print('Hourly profile shape:', hour_profile.shape)
print('Hottest day and sensor indexes:', (hot_day, hot_sensor))
print('Hottest day and sensor IDs:', (meta['days'][hot_day], meta['sensor_ids'][
```

```
Mean per sensor: [27.98941176 29.06385593 25.91185345 28.28786325 30.47807692 27.
2178903 ]
Hourly profile shape: (24,)
Hottest day and sensor indexes: (np.int64(9), np.int64(4))
Hottest day and sensor IDs: (np.int64(10), np.str_('T5'))
```

3d (4 marks) For sensor T3 (index 2) compute the 3-hour moving average of the hourly series for day 4 (index 3) using `np.cumsum` and slicing only, not `np.convolve`. Fill NaN in that series with the mean of the series first. Report the length of the result and explain why it is not 24.

In [110…
```
# YOUR CODE HERE
series = calibrated[3, :, 2].copy()
series[np.isnan(series)] = np.nanmean(series)
total = np.cumsum(np.insert(series, 0, 0))
moving_avg = (total[3:] - total[:-3]) / 3
print('Moving average:', moving_avg)
print('Length:', len(moving_avg))
```

```
Moving average: [19.2        18.89666667 18.56333333 18.79       19.53       20.5
3666667
 21.96666667 23.44333333 25.19       27.15666667 28.29       29.94333333
 28.7815942  29.88826087 30.23826087 32.31666667 31.93333333 30.54666667
 28.75333333 27.07333333 25.20666667 23.55333333]
Length: 22
```

3e (3 marks) Count the hours per day in which at least one sensor is above 35 degrees C (use `any`), and the number of days in which all sensors stay below 35 for the whole day (use `all`). Using `np.unique` with counts on the hour of each day's peak (use the sensor-averaged series), state which hour is the most common peak. Interpret in two sentences: is the sensor network behaving like a daytime heating cycle?

In [111…
```
# YOUR CODE HERE
above = calibrated > 35
hours_above = np.any(above, axis=2).sum(axis=1)
days_below = np.all(calibrated < 35, axis=(1, 2)).sum()
daily_sensor_avg = np.nanmean(calibrated, axis=2)
peak_hours = np.nanargmax(daily_sensor_avg, axis=1)
peak_values, peak_counts = np.unique(peak_hours, return_counts=True)
common_peak = peak_values[np.argmax(peak_counts)]
print('Hours with at least one sensor above 35 per day:', hours_above)
print('Days with all readings below 35:', days_below)
print('Peak hours:', peak_hours)
```

```
print('Peak hour counts:', dict(zip(peak_values, peak_counts)))
print('Most common peak hour:', common_peak)
```

```
Hours with at least one sensor above 35 per day: [1 3 5 7 5 7 8 7 9 9]
Days with all readings below 35: 0
Peak hours: [16 15 15 15 15 16 16 14 15 16]
Peak hour counts: {np.int64(14): np.int64(1), np.int64(15): np.int64(5), np.int64
(16): np.int64(4)}
Most common peak hour: 15
```

ANSWER 3e interpretation:

Question 4: Weather stations with Pandas (20 marks)

`data/weather_stations.csv` contains daily records for five cities. The file is messy: mixed date formats (`YYYY-MM-DD` and `DD/MM/YYYY`), temperatures stored as text such as `34.2C` , -999 as failed rain gauge value, missing humidity, pressure readings of 0 and duplicate rows.

4a (6 marks) Load and clean. Print `info()` before and after. Remove exact duplicates, parse both date formats into one datetime column (a plain `pd.to_datetime` call without care will silently misread days and months, check that), convert temperature to float, turn -999 into NaN, turn pressure 0 into NaN, fill missing humidity with the median humidity of the same city. Report the number of rows removed and the number of values you changed in each column. Name the cleaned frame `weather` .

In [112…

```
raw = pd.read_csv('data/weather_stations.csv')

weather = raw.drop_duplicates().copy()

weather['date'] = pd.to_datetime(weather['date'], format='mixed', dayfirst=True)

weather['temp_c'] = weather['temp_c'].astype(str).str.replace('C', '', regex=Fal

weather['rain_mm'] = weather['rain_mm'].replace(-999, np.nan)

weather['pressure_hpa'] = weather['pressure_hpa'].replace(0, np.nan)

weather['humidity_pct'] = weather.groupby('city')['humidity_pct'].transform(
    lambda x: x.fillna(x.median())
)

print(weather.info())
```

```
<class 'pandas.DataFrame'>
Index: 600 entries, 0 to 607
Data columns (total 7 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   station_id    600 non-null    str
 1   city          600 non-null    str
 2   date          600 non-null    datetime64[us]
 3   temp_c        600 non-null    float64
 4   humidity_pct  600 non-null    float64
 5   pressure_hpa  596 non-null    float64
 6   rain_mm       582 non-null    float64
dtypes: datetime64[us](1), float64(4), str(2)
memory usage: 37.5 KB
None
```

4b (5 marks) Create a month column. Build a pivot table of mean temperature by city (rows) and month (columns), rounded to 1 decimal. Find the city with the largest rise from the first month to the last month. Add a column giving each row's temperature anomaly relative to its own city's mean, using `groupby` with `transform`.

In [113…

```python
# YOUR CODE HERE
weather['month'] = weather['date'].dt.month

pivot = weather.pivot_table(
    index='city',
    columns='month',
    values='temp_c',
    aggfunc='mean'
).round(1)

rise = pivot.iloc[:, -1] - pivot.iloc[:, 0]

city = rise.idxmax()

weather['temp_anomaly'] = weather['temp_c'] - weather.groupby('city')['temp_c'].

print(pivot)
print(rise)
print(city)
print(weather[['city', 'temp_c', 'temp_anomaly']].head())
```

```
month        1     2     3     4     5     6     7     8     9     10    11  \
city
Islamabad  30.6  30.6  30.9  30.4  34.4  35.7  29.9  31.3  29.7  32.8  31.0
Karachi    33.7  33.0  33.6  32.7  37.6  39.2  32.4  33.9  36.1  36.0  33.7
Lahore     34.4  34.2  34.1  34.0  38.2  39.7  35.6  35.0  35.4  36.9  37.8
Peshawar   33.6  33.4  33.6  34.0  37.1  38.8  33.2  35.2  34.5  35.3  33.2
Quetta     24.4  23.7  24.2  24.3  29.3  29.2  26.8  26.0  29.0  24.8  25.7

month        12
city
Islamabad  32.8
Karachi    33.3
Lahore     35.3
Peshawar   36.8
Quetta     26.4
city
Islamabad    2.2
Karachi     -0.4
Lahore       0.9
Peshawar     3.2
Quetta       2.0
dtype: float64
Peshawar
        city  temp_c  temp_anomaly
0    Karachi    35.8      0.501667
1    Karachi    32.4     -2.898333
2  Islamabad    32.6      0.228333
3  Islamabad    28.6     -3.771667
4     Quetta    22.8     -3.711667
```

4c (5 marks) A heat spell is 3 or more consecutive days with temp above 38 in one city. Find every spell (city, start date, length). You may use NumPy on each city's sorted series, for example `diff` on a boolean array, but explain your logic in comments. If your data contains a missing calendar day for a city, say what you did about it.

In [114…

```python
spells = []
for city, group in weather.sort_values('date').groupby('city'):
    group = group.sort_values('date').reset_index(drop=True)
    hot = group['temp_c'].to_numpy() > 38
    dates = group['date'].to_numpy()
    start = None
    length = 0
    for i in range(len(group)):
        consecutive = i == 0 or (dates[i] - dates[i - 1]) == np.timedelta64(1, '
        # A run continues only when both readings are hot and dates are consecut
        if hot[i] and (length == 0 or consecutive):
            if length == 0:
                start = dates[i]
            length += 1
        else:
            if length >= 3:
                spells.append((city, pd.Timestamp(start), length))
            length = 1 if hot[i] else 0
            start = dates[i] if hot[i] else None
    if length >= 3:
        spells.append((city, pd.Timestamp(start), length))
print('Heat spells (city, start date, length):')
```

```
print(spells)
print('Missing calendar days were treated as breaks, so a spell must use consecu
```

```
Heat spells (city, start date, length):
[('Islamabad', Timestamp('2026-06-25 00:00:00'), 3), ('Karachi', Timestamp('2026-
05-12 00:00:00'), 5), ('Karachi', Timestamp('2026-05-20 00:00:00'), 3), ('Karach
i', Timestamp('2026-06-18 00:00:00'), 11), ('Lahore', Timestamp('2026-05-16 00:0
0:00'), 7), ('Lahore', Timestamp('2026-06-16 00:00:00'), 7), ('Lahore', Timestamp
('2026-06-24 00:00:00'), 5), ('Peshawar', Timestamp('2026-05-15 00:00:00'), 6),
('Peshawar', Timestamp('2026-06-17 00:00:00'), 4), ('Peshawar', Timestamp('2026-0
6-22 00:00:00'), 7)]
Missing calendar days were treated as breaks, so a spell must use consecutive dat
es.
```

4d (4 marks) Compute the correlation matrix of temperature, humidity, pressure and rain. Then compute it separately for Karachi only. Interpret: which relationships are weak overall but strong inside one city, and why could pooling cities mislead you (name the effect)? Rain is mostly zero, so also state what that does to a correlation coefficient.

In [115…

```
# YOUR CODE HERE
spells = []
for city, group in weather.groupby('city'):
    group = group.sort_values('date').reset_index(drop=True)
    start = None
    count = 0
    for i in range(len(group)):
        hot = group.loc[i, 'temp_c'] > 38
        if i > 0:
            consecutive = (group.loc[i, 'date'] - group.loc[i-1, 'date']).days =
        else:
            consecutive = True
        if hot and (count == 0 or consecutive):
            if count == 0:
                start = group.loc[i, 'date']
            count += 1
        else:
            if count >= 3:
                spells.append((city, start, count))
            if hot:
                start = group.loc[i, 'date']
                count = 1
            else:
                start = None
                count = 0
    if count >= 3:
        spells.append((city, start, count))
print(spells)
```

```
[('Islamabad', Timestamp('2026-06-25 00:00:00'), 3), ('Karachi', Timestamp('2026-
05-12 00:00:00'), 5), ('Karachi', Timestamp('2026-05-20 00:00:00'), 3), ('Karach
i', Timestamp('2026-06-18 00:00:00'), 11), ('Lahore', Timestamp('2026-05-16 00:0
0:00'), 7), ('Lahore', Timestamp('2026-06-16 00:00:00'), 7), ('Lahore', Timestamp
('2026-06-24 00:00:00'), 5), ('Peshawar', Timestamp('2026-05-15 00:00:00'), 6),
('Peshawar', Timestamp('2026-06-17 00:00:00'), 4), ('Peshawar', Timestamp('2026-0
6-22 00:00:00'), 7)]
```

ANSWER 4d interpretation:

Question 5: Student performance (15 marks)

`data/student_performance.csv` has 240 students (some rows duplicated). Maximum marks are in the column names: quiz 20, assignment 20, midterm 25, final 35. Total = 100. Some midterms are the text `Absent`, some finals look like a typing slip (entered ten times too large), and some study hours are missing.

5a (5 marks) Clean the data. Absent counts as 0 for the total but must be kept in a separate boolean column. Fix finals that exceed 35 by the most defensible rule and say what rule you chose. Fill missing study hours with the section median. Create `total` and a `grade` column: A at 85 or more, B at 70 to 84.99, C at 55 to 69.99, D at 40 to 54.99, F below 40, using `np.where` (nested) or boolean masks, not a loop.

In [116…

```
raw_students = pd.read_csv('data/student_performance.csv')
students = raw_students.drop_duplicates().copy()
students['absent_midterm'] = students['midterm_25'].eq('Absent')
students['midterm_25'] = pd.to_numeric(students['midterm_25'], errors='coerce').
students['final_35'] = np.where(students['final_35'] > 35, students['final_35']
students['study_hours_week'] = students.groupby('section')['study_hours_week'].t
    lambda values: values.fillna(values.median())
)
students['total'] = students['quiz_20'] + students['assignment_20'] + students['
students['grade'] = np.where(students['total'] >= 85, 'A',
                    np.where(students['total'] >= 70, 'B',
                    np.where(students['total'] >= 55, 'C',
                    np.where(students['total'] >= 40, 'D', 'F'))))
print('Rows before:', len(raw_students))
print('Rows after removing duplicates:', len(students))
print('Duplicate rows removed:', len(raw_students) - len(students))
print('Absent midterms:', students['absent_midterm'].sum())
print('Final marks corrected:', (raw_students.loc[students.index, 'final_35'] >
print('Missing study hours filled:', raw_students.loc[students.index, 'study_hou
print(students.head())
```

```
Rows before: 243
Rows after removing duplicates: 240
Duplicate rows removed: 3
Absent midterms: 9
Final marks corrected: 4
Missing study hours filled: 7
  student_id section  attendance_pct  study_hours_week  quiz_20  \
0      S1000       A            81.0               7.4     11.7
1      S1001       C            71.0              10.1     11.8
2      S1002       B            95.0              13.7     11.0
3      S1003       B            55.0               8.8      7.0
4      S1004       A            70.0               7.8      9.8

   assignment_20  midterm_25  final_35  absent_midterm  total grade
0           12.4         8.6      26.2           False   58.9     C
1           15.3        14.0      24.5           False   65.6     C
2           20.0        16.8      25.4           False   73.2     B
3           11.6        13.4      13.0           False   45.0     D
4            9.0        10.9      21.9           False   51.6     D
```

5b (5 marks) Per section report: number of students, mean total, std of total, pass percentage (not F), and the student_id with the highest total. Then list students with attendance below 75 who still scored a B or better. Use a single boolean mask.

In [117…

```
section_report = students.groupby('section').agg(
    students=('student_id', 'count'),
    mean_total=('total', 'mean'),
    std_total=('total', 'std'),
    pass_percentage=('grade', lambda values: (values != 'F').mean() * 100)
)
top_ids = students.loc[students.groupby('section')['total'].idxmax(), ['section'
section_report = section_report.join(top_ids.set_index('section')['student_id'].
print(section_report)
mask = (students['attendance_pct'] < 75) & (students['grade'].isin(['A', 'B']))
print('Attendance below 75 and grade B or better:')
print(students.loc[mask, ['student_id', 'section', 'attendance_pct', 'total', 'g
```

```
         students  mean_total  std_total  pass_percentage top_student_id
section
A              71   63.088732  14.688096        91.549296          S1074
B              64   62.946875  13.460158        96.875000          S1082
C              49   65.067347  12.455126        97.959184          S1017
D              56   64.487500  15.130392        92.857143          S1055
Attendance below 75 and grade B or better:
    student_id section  attendance_pct  total grade
5        S1005       A            60.0   73.6     B
9        S1009       D            65.0   77.2     B
66       S1066       A            71.0   77.3     B
89       S1089       B            74.0   81.3     B
92       S1092       A            68.0   76.4     B
106      S1106       A            70.0   72.8     B
126      S1126       A            67.0   72.2     B
138      S1138       B            72.0   75.7     B
139      S1139       A            67.0   75.1     B
176      S1176       A            71.0   77.8     B
177      S1177       C            73.0   88.8     A
209      S1209       A            65.0   70.1     B
```

5c (5 marks) Compute the correlation of study hours with total and with final. Fit a straight line of total on study hours with `np.polyfit` (degree 1) and report slope and intercept. Interpret the slope in plain words with units. Then compare mean total of absent-midterm students and everyone else and state whether this proves absence causes low marks. Mention at least one confounder visible in the data.

In [118…

```
hours_total_corr = students['study_hours_week'].corr(students['total'])
hours_final_corr = students['study_hours_week'].corr(students['final_35'])
slope, intercept = np.polyfit(students['study_hours_week'], students['total'], 1
absent_mean = students.loc[students['absent_midterm'], 'total'].mean()
present_mean = students.loc[~students['absent_midterm'], 'total'].mean()
print('Study hours and total correlation:', hours_total_corr)
print('Study hours and final correlation:', hours_final_corr)
print('Slope:', slope)
print('Intercept:', intercept)
print('Mean total, absent midterm:', absent_mean)
print('Mean total, present midterm:', present_mean)
print('This does not prove absence causes low marks. Attendance and study hours
```

```
Study hours and total correlation: 0.7766228955402087
Study hours and final correlation: 0.6806658662093116
Slope: 2.171242586535732
Intercept: 37.72317256613251
Mean total, absent midterm: 50.355555555555554
Mean total, present midterm: 64.304329004329
This does not prove absence causes low marks. Attendance and study hours are poss
ible confounders.
```

ANSWER 5c interpretation:

Question 6: Dashboard (15 marks)

Build one figure, `figsize=(14, 9)`, using `GridSpec` with 3 rows and 3 columns and constrained layout or `tight_layout`. Use your cleaned `weather`, `students` (from Q5) and the calibrated sensor cube. Required panels:

1. (top row, spans 2 columns) Daily mean temperature of each city as lines with different line styles, a shared legend outside the axes and a marked heat spell using `fill_between` or `axvspan`. Add a twin y-axis with mean humidity of one city (a different colour, with colour matched tick labels).
2. (top right) Horizontal bar chart of mean total per section with `errorbar` style whiskers showing std. Print the bar values as text.
3. (middle row, left) Histogram of student totals with bins every 5 marks from 0 to 100, a vertical dashed line at the mean, and `ax.annotate` with an arrow pointing to the tallest bin. Use data coordinates for the arrow target and axes coordinates for the text.
4. (middle row, middle) Scatter of study hours vs total coloured by attendance, with colorbar and the polyfit line from Q5c.
5. (middle right) Step plot of the hourly sensor-average profile (24 hours), y-axis limits chosen by you and justified in a comment.
6. (bottom row, spans all columns) `imshow` heatmap of the day by hour mean sensor temperature (10 x 24) with a diverging colormap centred on the overall mean using `vmin` / `vmax`, tick labels showing real day numbers, and an inset axes (`inset_axes` or `fig.add_axes`) showing the hour with the maximum anomaly as a small bar chart over days. Mark split: layout 3, panels 1 to 2 4, panels 3 to 5 4, panel 6 3, labels/titles/legend quality 1. Save as PNG at 200 dpi and as PDF.

In [123…

```python
# YOUR CODE HERE
import os
from mpl_toolkits.axes_grid1.inset_locator import inset_axes
os.makedirs('outputs', exist_ok=True)
fig = plt.figure(figsize=(14, 9), constrained_layout=True)
gs = GridSpec(3, 3, figure=fig)
ax1 = fig.add_subplot(gs[0, 0:2])
styles = ['-', '--', '-.', ':', (0, (3, 1, 1, 1))]
for style, (city, group) in zip(styles, weather.groupby('city')):
    daily = group.groupby('date')['temp_c'].mean().sort_index()
    ax1.plot(daily.index, daily.values, linestyle=style, label=city)
ax1.set_title('Daily mean temperature by city')
ax1.set_ylabel('Temperature (°C)')
```

```
ax1.tick_params(axis='x', rotation=25)


hot_city = 'Karachi'
humidity_daily = weather[weather['city'] == hot_city].groupby('date')['humidity_
ax1b = ax1.twinx()
ax1b.plot(humidity_daily.index, humidity_daily.values, color='tab:green', alpha=
ax1b.set_ylabel('Karachi humidity (%)', color='tab:green')
ax1b.tick_params(axis='y', labelcolor='tab:green')
handles1, labels1 = ax1.get_legend_handles_labels()
handles2, labels2 = ax1b.get_legend_handles_labels()
fig.legend(handles1 + handles2, labels1 + labels2, loc='upper center', bbox_to_a
if spells:
    spell_city, spell_start, spell_length = spells[0]
    spell_dates = pd.date_range(spell_start, periods=spell_length, freq='D')
    ax1.axvspan(spell_dates[0], spell_dates[-1] + pd.Timedelta(days=1), color='c

ax2 = fig.add_subplot(gs[0, 2])
section_means = students.groupby('section')['total'].mean()
section_std = students.groupby('section')['total'].std()
y_pos = np.arange(len(section_means))
ax2.barh(y_pos, section_means.values, alpha=0.8)
ax2.errorbar(section_means.values, y_pos, xerr=section_std.values, fmt='none', c
ax2.set_yticks(y_pos, section_means.index)
ax2.set_title('Mean total by section')
ax2.set_xlabel('Marks')
for y, value in zip(y_pos, section_means.values):
    ax2.text(value + 0.5, y, f'{value:.1f}', va='center', fontsize=8)

ax3 = fig.add_subplot(gs[1, 0])
bins = np.arange(0, 105, 5)
counts, edges, patches = ax3.hist(students['total'], bins=bins, edgecolor='black
mean_total = students['total'].mean()
ax3.axvline(mean_total, linestyle='--', color='black', label=f'Mean = {mean_tota
ax3.set_title('Distribution of student totals')
ax3.set_xlabel('Total marks')
ax3.set_ylabel('Students')
tallest = np.argmax(counts)
target_x = (edges[tallest] + edges[tallest + 1]) / 2
target_y = counts[tallest]
ax3.annotate('Tallest bin', xy=(target_x, target_y), xycoords='data',
             xytext=(0.68, 0.88), textcoords='axes fraction',
             arrowprops={'arrowstyle': '->'})
ax3.legend()

ax4 = fig.add_subplot(gs[1, 1])
points = ax4.scatter(students['study_hours_week'], students['total'],
                     c=students['attendance_pct'], cmap='viridis')
line_x = np.linspace(students['study_hours_week'].min(), students['study_hours_w
ax4.plot(line_x, slope * line_x + intercept, color='red')
fig.colorbar(points, ax=ax4, label='Attendance (%)')
ax4.set_title('Study hours and total')
ax4.set_xlabel('Study hours per week')
ax4.set_ylabel('Total marks')

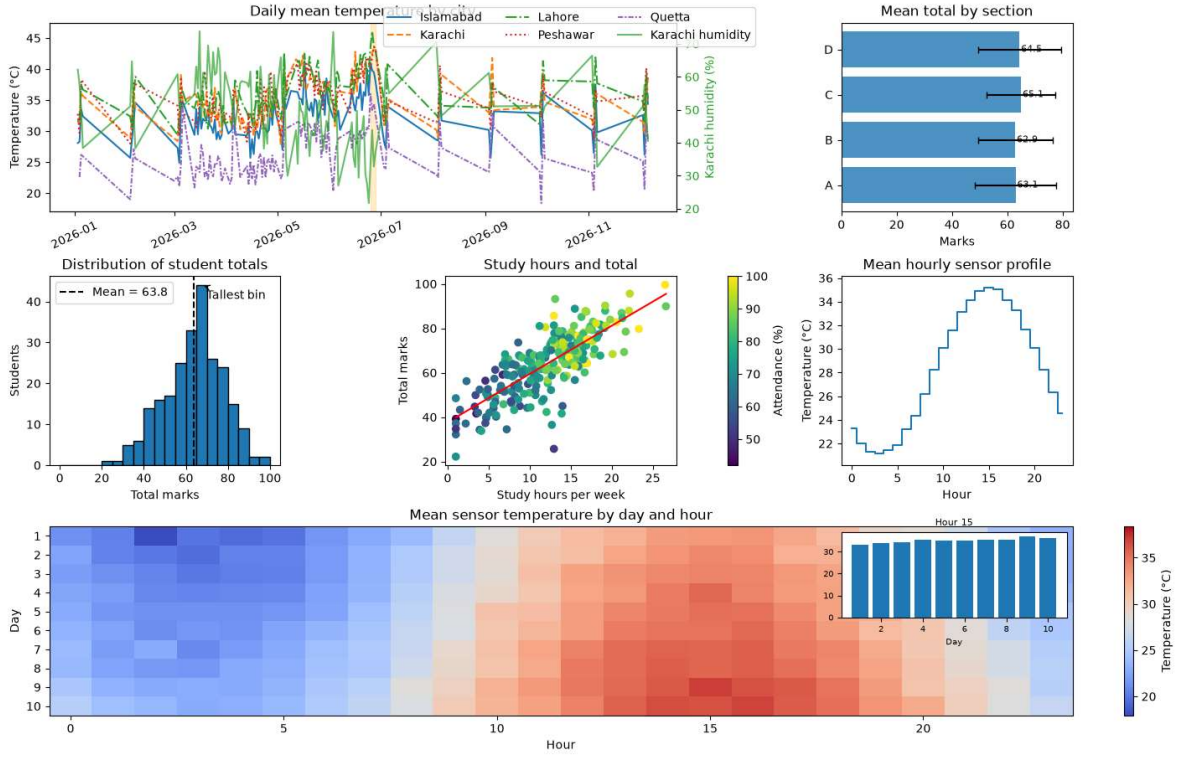
ax5 = fig.add_subplot(gs[1, 2])
ax5.step(np.arange(24), hour_profile, where='mid')
ax5.set_title('Mean hourly sensor profile')
ax5.set_xlabel('Hour')
ax5.set_ylabel('Temperature (°C)')
```

```
ax5.set_ylim(np.nanmin(hour_profile) - 1, np.nanmax(hour_profile) + 1)  # A smal

ax6 = fig.add_subplot(gs[2, :])
day_hour = np.nanmean(calibrated, axis=2)
overall_mean = np.nanmean(day_hour)
spread = np.nanmax(np.abs(day_hour - overall_mean))
im = ax6.imshow(day_hour, aspect='auto', cmap='coolwarm',
                vmin=overall_mean - spread, vmax=overall_mean + spread)
ax6.set_title('Mean sensor temperature by day and hour')
ax6.set_xlabel('Hour')
ax6.set_ylabel('Day')
ax6.set_yticks(np.arange(10), meta['days'])
fig.colorbar(im, ax=ax6, label='Temperature (°C)')
hour_anomaly = hour_profile - overall_mean
max_hour = np.argmax(hour_anomaly)
small_ax = inset_axes(ax6, width='22%', height='45%', loc='upper right')
small_ax.bar(meta['days'], day_hour[:, max_hour])
small_ax.set_title(f'Hour {max_hour}', fontsize=8)
small_ax.set_xlabel('Day', fontsize=7)
small_ax.tick_params(labelsize=7)

fig.savefig('outputs/dashboard.png', dpi=200)
fig.savefig('outputs/dashboard.pdf')
#plt.close(fig)
print('Created dashboard.png and dashboard.pdf')
print('Overall sensor mean:', overall_mean)
print('Maximum anomaly hour:', max_hour)
print('First marked heat spell:', spells[0] if spells else 'No heat spell found'
```

```
Created dashboard.png and dashboard.pdf
Overall sensor mean: 28.15007222222222
Maximum anomaly hour: 15
First marked heat spell: ('Islamabad', Timestamp('2026-06-25 00:00:00'), 3)
```



ANSWER 6 (4 sentences): Using specific numbers from your dashboard, tell a plant manager what the single most important finding is, and name one panel whose design could mislead a reader and why.

Question 7: Surfaces and animation (10 marks)

Let f(x, y) = sin(r) / r with r = sqrt(x^2 + y^2) on x, y in [-10, 10].

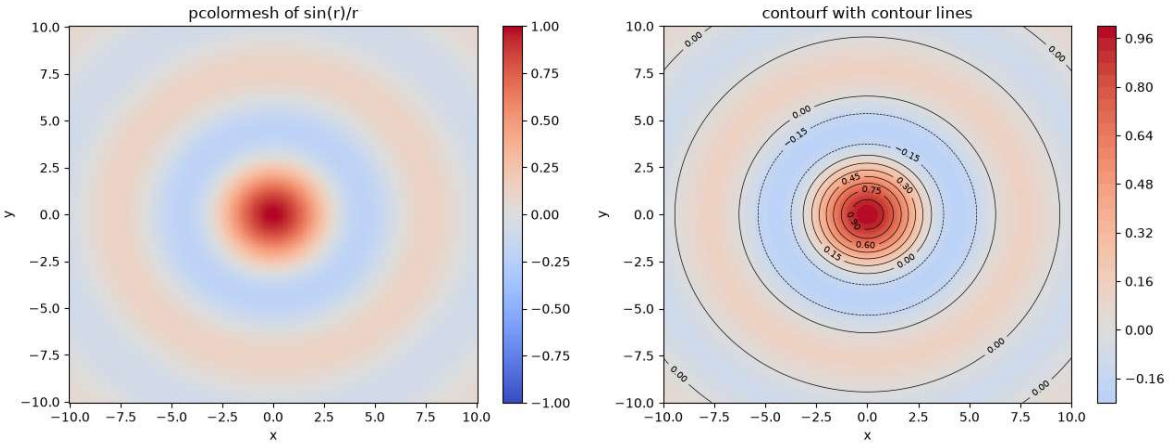
7a (4 marks) Build the grid with `meshgrid` of 201 points per axis. Handle r = 0 correctly (the limit is 1; a plain division gives NaN, show you saw it and fixed it with `np.where`). Draw side by side: `pcolormesh` with a diverging colormap and symmetric `vmin` / `vmax`, and `contourf` with `contour` lines labelled using `clabel`. Add colorbars. Count (with Boolean sums) how many grid points have f below -0.1 and say what visual feature that corresponds to.

In [124…

```
# YOUR CODE HERE
x = np.linspace(-10, 10, 201)
y = np.linspace(-10, 10, 201)
X, Y = np.meshgrid(x, y)
R = np.sqrt(X**2 + Y**2)
with np.errstate(divide='ignore', invalid='ignore'):
    Z = np.sin(R) / R
print('NaN at the centre before fixing:', np.isnan(Z).sum())
Z = np.where(R == 0, 1, Z)
limit = np.max(np.abs(Z))
below_count = np.sum(Z < -0.1)

fig, ax = plt.subplots(1, 2, figsize=(13, 5))
one = ax[0].pcolormesh(X, Y, Z, cmap='coolwarm', vmin=-limit, vmax=limit, shadin
fig.colorbar(one, ax=ax[0])
ax[0].set_title('pcolormesh of sin(r)/r')
two = ax[1].contourf(X, Y, Z, levels=30, cmap='coolwarm', vmin=-limit, vmax=limi
lines = ax[1].contour(X, Y, Z, levels=10, colors='black', linewidths=0.5)
ax[1].clabel(lines, inline=True, fontsize=7)
fig.colorbar(two, ax=ax[1])
ax[1].set_title('contourf with contour lines')
for a in ax:
    a.set_xlabel('x')
    a.set_ylabel('y')
fig.tight_layout()
#plt.close(fig)
print('Grid shape:', Z.shape)
print('Grid points below -0.1:', below_count)
```

```
NaN at the centre before fixing: 1
Grid shape: (201, 201)
Grid points below -0.1: 6280
```



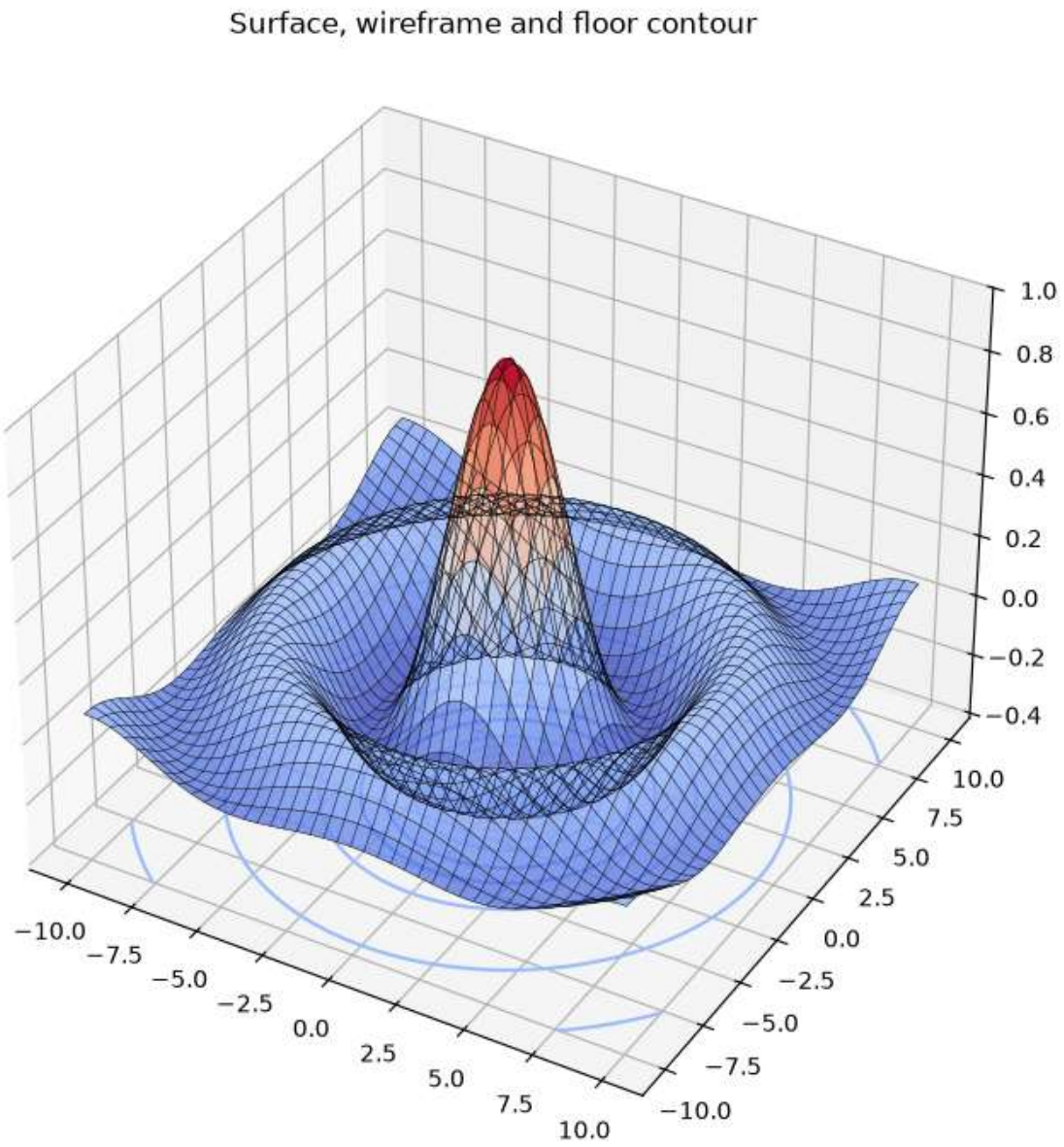
7b (3 marks) Make a 3D surface with `plot_surface` using `rstride=5, cstride=5`, then a `plot_wireframe` of the same data, and add a contour projection onto the floor (`zdir="z"`, with offset below the minimum). Use `view_init(elev=35, azim=-60)`. Explain what changes if rstride and cstride are set to 1 and why.

In [125…

```
# YOUR CODE HERE
from mpl_toolkits.mplot3d import Axes3D

fig = plt.figure(figsize=(9, 7))
ax = fig.add_subplot(111, projection='3d')
ax.plot_surface(X, Y, Z, cmap='coolwarm', rstride=5, cstride=5, alpha=0.8)
ax.plot_wireframe(X, Y, Z, rstride=5, cstride=5, color='black', linewidth=0.35)
ax.contour(X, Y, Z, zdir='z', offset=np.nanmin(Z) - 0.2, cmap='coolwarm')
ax.set_zlim(np.nanmin(Z) - 0.2, np.nanmax(Z))
ax.view_init(elev=35, azim=-60)
ax.set_title('Surface, wireframe and floor contour')
plt.tight_layout()
#plt.close(fig)
print('With rstride=1 and cstride=1, every grid row and column is drawn, making
```

```
With rstride=1 and cstride=1, every grid row and column is drawn, making the surf
ace and wireframe denser and slower to render.
```



7c (3 marks) Animate the travelling wave sin(r - 0.3 t)/r along the line y = 0 using `FuncAnimation` with 60 frames, `interval=50` and `blit=True`. Your update function must return the artist(s). Save as `wave.gif` with `PillowWriter`. Do not use `plt.show()` blocking calls. Say in a comment why blit needs the update function to return a tuple.

In [126…

```
# YOUR CODE HERE
line_x = np.linspace(-10, 10, 201)
line_r = np.abs(line_x)
fig, ax = plt.subplots(figsize=(8, 4))
wave_line, = ax.plot(line_x, np.zeros_like(line_x))
ax.set_xlim(-10, 10)
ax.set_ylim(-10, 10)
ax.set_xlabel('x')
ax.set_ylabel('Wave value')
ax.set_title('Travelling radial wave along y = 0')

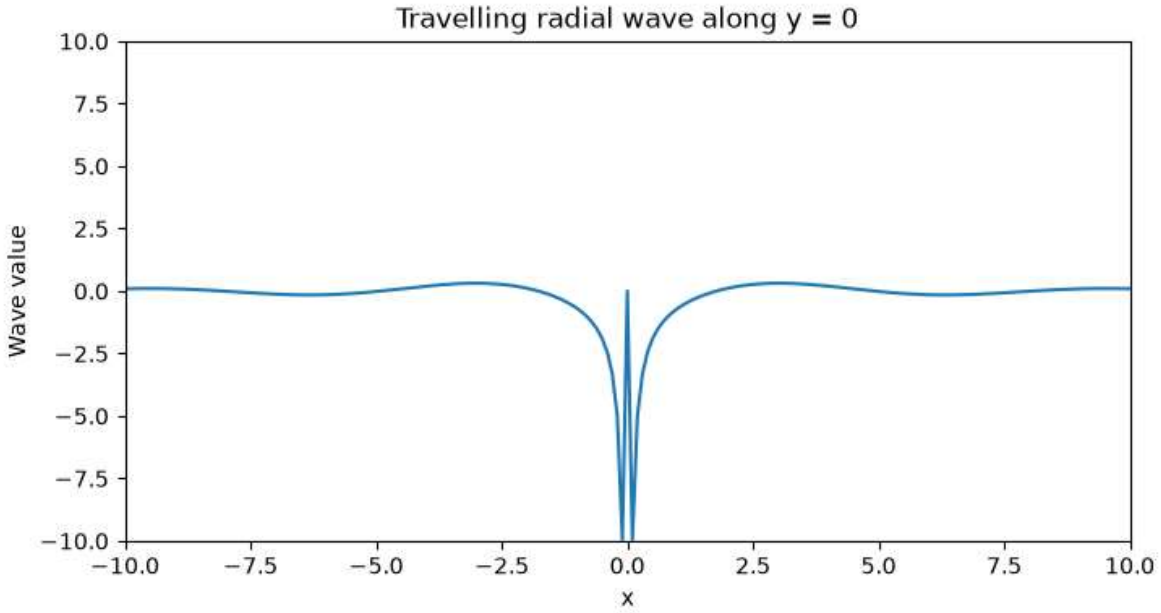
def update(frame):
    t = frame * 0.1
    # The formula is singular at x=0 for t>0, so that plotted sample is set to z
    values = np.divide(np.sin(line_r - 0.3 * t), line_r,
                       out=np.zeros_like(line_r), where=line_r != 0)
```

```
    wave_line.set_ydata(values)
    return (wave_line,)  # blit needs the changed artist returned so it can redr

animation = FuncAnimation(fig, update, frames=60, interval=50, blit=True)
os.makedirs('outputs', exist_ok=True)
animation.save('outputs/wave.gif', writer=PillowWriter())
#plt.close(fig)
print('Created wave.gif with 60 frames.')
from IPython.display import Image
Image(filename='outputs/wave.gif')
```

```
Created wave.gif with 60 frames.
```

Out[126… <IPython.core.display.Image object>



Final checklist

- Restart and Run All works
- All ANSWER cells filled
- `dashboard.png`, `dashboard.pdf` and `wave.gif` are created by your notebook
- Code pushed to GitHub as described in the PDF